

Chapitre 4 : Commandes en C, Appels système fichiers – TD/TP

Table des matières

1	Travaux Dirigés	1
1.1	mon_getenv.c	1
1.2	copier.c	2
2	Travaux Pratiques	2
2.1	mon_echo.c	2
2.2	mon_test.c	3
2.3	mon_wc.c	3
2.4	mon_cat.c	3
2.5	mon_cp.c	3

1 Travaux Dirigés

Les exercices suivants vous font utiliser les fonctions de manipulation de chaîne et les appels système pour les entrées/sorties fichiers. Vous devez les développer et les tester dans votre environnement de développement.

1.1 mon_getenv.c

Réimplémente la fonction `getenv(3)` en utilisant la variable globale externe `char** environ` et teste la fonction avec différents noms de variable passés en argument du programme principal.

Le programme utilise une ou plusieurs fonctions de manipulation de chaîne suivantes : `strlen(3)`, `strcmp(3)`, `strstr(3)` ou `strchr(3)`.

Remarque : la variable globale externe `environ` pourrait être déclarée de façon équivalente `char* environ[]`, faisant ressortir qu'il s'agit d'un tableau de chaînes de caractères. Le tableau est terminé par `NULL` et les chaînes sont de la forme `VARIABLE=valeur`.

Testez votre code dans plusieurs cas, comme par exemple :

```
$ ./mon_getenv HOME foo PWD SHELL
HOME=/home/lamia
foo=(null)
PWD=/home/lamia/ISYS-TD4
```

```
SHELL=/usr/bin/bash
$
```

1.2 copier.c

Copie le fichier spécifié en premier argument dans le fichier spécifié en deuxième argument. Si le second fichier n'existe pas déjà, il est créé. S'il existe déjà, son contenu est supprimé.

Le programme utilise les appels systèmes `open(2)`, `read(2)`, `write(2)` et `close(2)`.

Testez que le fichier copie est bien identique à l'original, comme par exemple :

```
$ ./copier /etc/passwd foo
$ diff /etc/passwd foo
$ echo $?
0
$
```

2 Travaux Pratiques

Il s'agit de développer en C les programmes spécifiés dans les paragraphes suivants. Ces programmes doivent respecter les conventions vues en cours :

- Les messages normaux sont écrits sur la sortie standard (descripteur de fichier 1, ou stdout), et les messages d'erreur sur la sortie erreur (descripteur de fichier 2, ou stderr).
- Le statut d'exécution des scripts est 0 en cas de succès, et un nombre positif en cas d'erreur. En fonction de l'exercice, ce choix est libre ou imposé.

Les exercices consistent à réimplémenter des commandes existantes, parfois avec quelques-unes de leurs options simples à traiter. Consultez le man de ces commandes pour avoir leur synopsis et descriptions détaillées.

Le testeur automatique compile votre code avec les options suivantes :

`-Wall -Wextra -Werr -pedantic`

Veillez en conséquence à ne laisser aucune erreur ou avertissement dans votre code.

2.1 mon_echo.c

Réimplémente en C la commande `echo`. Les options `-n` et `-E` sont traitées, l'option `-e` est acceptée, i.e. elle ne génère pas d'erreur, mais elle n'est pas traitée.

Cet exercice est l'occasion d'observer expérimentalement le comportement de la commande `echo` et compléter les informations du man : qu'est-ce que la commande affiche et quel statut d'exécution retourne-t-elle lors des appels suivants ? Reproduisez ce comportement à l'identique.

\$ echo	# pas d'argument
\$ echo foo -n	# option valide après argument
\$ echo -n -x -E foo bar	# option invalide au milieu

```
$ echo -x -n -E foo bar baz      # option invalide au début
```

```
$ echo -n -n -E foo bar baz      # option spécifiée deux fois
```

Important : une fois votre commande mise au point, appelez-la avec différents développements (accolade, chemin, commandes, arithmétique...) pour vous convaincre que c'est bien le shell qui réalise ces développements avant d'appeler la commande, et non la commande elle-même.

2.2 mon_test.c

Réimplémente en C la commande `test` pour la partie test des chaînes de caractères, en prenant en compte les opérateurs unaires `-n` et `-z` ainsi que les opérateurs binaires `=`, `!=`, `<` et `>`.

Comme pour l'exercice précédent, vérifiez expérimentalement comment `test` se comporte dans différents cas d'erreur et reproduisez ce comportement.

Les messages d'erreur de votre commande doivent commencer par `"mon_test:"` et contenir l'argument qui cause l'erreur, ou l'information « pas assez d'arguments » ou « trop d'arguments ».

Important : une fois votre commande mise au point, appelez-la dans une construction `if` pour vérifier qu'elle s'interface correctement avec le shell. Exemple :

```
$ if ./mon_test $HOME = ~; then echo OK; else echo NOK; fi
OK
$
```

2.3 mon_wc.c

Réimplémente en C la commande `wc` sans aucune option. La commande sait traiter un ou plusieurs fichiers passés en argument, dont le nom spécial `'-'`, ou l'entrée standard quand aucun n'est spécifié. La commande lit les fichiers à l'aide des appels système `open(2)`, `read(2)` et `close(2)`.

Conseils d'implémentation : Définissez une fonction C qui calcule les statistiques complètes d'un fichier seul, fonction que `main()` appelle sur chacun des fichiers à traiter. Utilisez des variables globales pour stocker les statistiques totales que `wc` affiche sur plusieurs fichiers.

Important : une fois votre commande mise au point, vérifiez qu'elle produit exactement les mêmes valeurs, présentés de la même façon, que la commande `wc`.

2.4 mon_cp.c

Réimplémente en C la commande `cp` sans aucune option, en traitant les deux formes suivantes, décrites dans le man :

```
cp SOURCE DEST
```

```
cp SOURCE... DIRECTORY
```

Pour simplifier le code, le `DEST` de la première forme doit être un chemin d'accès complet : ce ne peut être un simple répertoire. La commande lit et écrit les fichiers à l'aide des appels système `open(2)`, `read(2)`, `write(2)` et `close(2)`.

Conseils d'implémentation : définissez une fonction C qui copie un fichier vers un autre, fonction que `main()` appelle sur chacun des fichiers à traiter. La syntaxe `SOURCE... DIRECTORY` implique des manipulations sur les chemins d'accès aux entrées : les fonctions `snprintf(3)` et `strchr(3)` vous seront peut-être utiles.

Important : une fois votre commande mise au point, vérifiez à l'aide des commandes `diff` ou `cmp` que la copie d'un fichier générée par votre commande est strictement identique à l'original.

2.5 mon_cat.c

Réimplémente en C la commande `cat` avec l'option `-E`. La commande sait traiter 0, 1 ou plusieurs arguments. Elle lit les fichiers à l'aide des appels système `open(2)`, `read(2)` et `close(2)`.

Conseils d'implémentation : Définissez une fonction C qui affiche le contenu d'un fichier seul, fonction que `main()` appelle sur chacun des fichiers à traiter. Concernant le traitement de l'option `-E`, vous pouvez :

- soit localiser les retours à la ligne avec la fonction `strchr(3)` ou `strchrnul(3)`, et écrire les lignes par blocs de caractères,
- soit développer un code plus simple et non optimisé consistant à écrire un par un les caractères d'une ligne.

Important : une fois votre commande mise au point, vérifiez à l'aide des redirections shell et des commandes `diff` ou `cmp` que l'affichage produit par votre commande est strictement identique à celui de la commande `cat`.

2.6 Remise

Remettez svp vos fichiers `.c` dans l'espace Moodle prévu à cet effet.

Vous devez les remettre à *plat*, c'est-à-dire sans répertoires ou sous-répertoires intermédiaires et sans les regrouper en archive. Il vous suffit de faire un simple glisser-déposer des différents fichiers depuis votre répertoire de développement dans l'espace Moodle.

Vous pouvez remettre vos fichiers autant de fois que vous le voulez jusqu'à la date limite. Toute nouvelle version doit *écraser* la précédente : les fichiers nommés par exemple `prog(1).c` seront ignorés lors de la correction.